Algorithmique & Programmation

Corwin FEVRE corwin.fevre@eigsi.fr

FISA 3 — EIGSI



Inspiré du cours de M. Thomas RAIMBAULT

Partie IV

Algorithmes de tri

1. Préambule
2. Trier une liste

1 Préambule
- Opération élémentaire : la permutation
- Humain versus Machine

2 Trier une liste
- Classes d'algorithmes
- Tris quadratiques en temps
- Autres tris

1 Préambule
- Opération élémentaire : la permutation
- Humain versus Machine

2 Trier une liste
- Classes d'algorithmes
- Tris quadratiques en temps
- Autres tris

Permuter 2 valeurs

- Rappel du principe
  - ⇒ **besoin d'une variable temporaire**, pour stocker une des valeurs permutées.

Code

```
1 liste =  [20, 40, 80, 60, 50]
2 print(liste)
3
4 tmp = liste[2]
5 liste[2]  = liste[4]
6 liste[4]  = tmp
7
8 print(liste)
```

Exemple (Affichage)

20, 40, 80, 60, 50
20, 40, 50, 60, 80

Fonction de permutation de 2 valeurs

- On suppose la fonction `permuter` disponible :
  - ✎ L'action principale est la permutation.
    → la fonction pourrait donc ne rien retourner
  - ✎ L'action secondaire est d'informer si une permutation a été effectuée.
    → `return` pour pouvoir enchaîner les actions.

Code (Définition)

```
1  def permuter(liste, index1, index2) :
2
3      permutationReussie = false
4
5      if liste :
6          if ((index1 >= 0) and (index1 < len(liste)) and
7              ((index2 >= 0) and (index2 < len(liste)) :
8
9              tmp = liste[index1]
10             liste[index1] = liste[index2]
11             liste[index2] = tmp
12             permutationReussie = true
13
14     return permutationReussie
```

Fonction de permutation de 2 valeurs

- On suppose la fonction `permuter` disponible :

Code (Utilisation)

```
1  liste =  [20, 40, 80, 60, 50]
2  print(liste)
3
4  ok = permuter(liste, 2, 4)
5  # Note : la valeur retournée permet d'enchaîner les actions
6
7  if ok :
8      print(liste)
9  else :
10     print("pas de modification de la liste")
```

Exemple (Affichage)

20, 40, 80, 60, 50
20, 40, 50, 60, 80

1 Préambule
- Opération élémentaire : la permutation
- Humain versus Machine

2 Trier une liste
- Classes d'algorithmes
- Tris quadratiques en temps
- Autres tris

Considérons la liste à trier suivante :

50	80	60	20	40

Considérons la liste à trier suivante : | 50 | 80 | 60 | 20 | 40 |

- l'**humain** voit :

Considérons la liste à trier suivante : | 50 | 80 | 60 | 20 | 40 |

- l'**humain** voit :
  - ⇒ **vision globale** 
    - | 50 | 80 | 60 | 20 | 40 |

Considérons la liste à trier suivante :

50	80	60	20	40

- l'**humain** voit :
  - ⇒ **vision globale** 
    - | 50 | 80 | 60 | 20 | 40 |

    - Quid si liste de grande taille... ?

50	80	60	20	40	10	90
25	39	81	63	72	91	42
08	14	63	25	02	85	73
99	01	06	33	22	28	66
01	26	48	57	88	36	29

Considérons la liste à trier suivante :

50	80	60	20	40

- l'**humain** voit :
  - ⇒ **vision globale** 
    - | 50 | 80 | 60 | 20 | 40 |
    - Quid si liste de grande taille... ?

50	80	60	20	40	10	90
25	39	81	63	72	91	42
08	14	63	25	02	85	73
99	01	06	33	22	28	66
01	26	48	57	88	36	29

⇒ Difficile à trier à la main... Utilisons une machine !

Considérons la liste à trier suivante : | 50 | 80 | 60 | 20 | 40 |

- la **machine** « voit » :

Considérons la liste à trier suivante : | 50 | 80 | 60 | 20 | 40 |

- la **machine** « voit » :

  $\Rightarrow$ **à travers un (des) index** !

  e.g. avec un index :

  | 50 | ?? | ?? | ?? | ?? | (index = 0)

  △

Considérons la liste à trier suivante : | 50 | 80 | 60 | 20 | 40 |

- la **machine** « voit » :
  - $\Rightarrow$ **à travers un (des) index** !

e.g. avec un index :

| 50 | ?? | ?? | ?? | ?? | (index = 0)

△

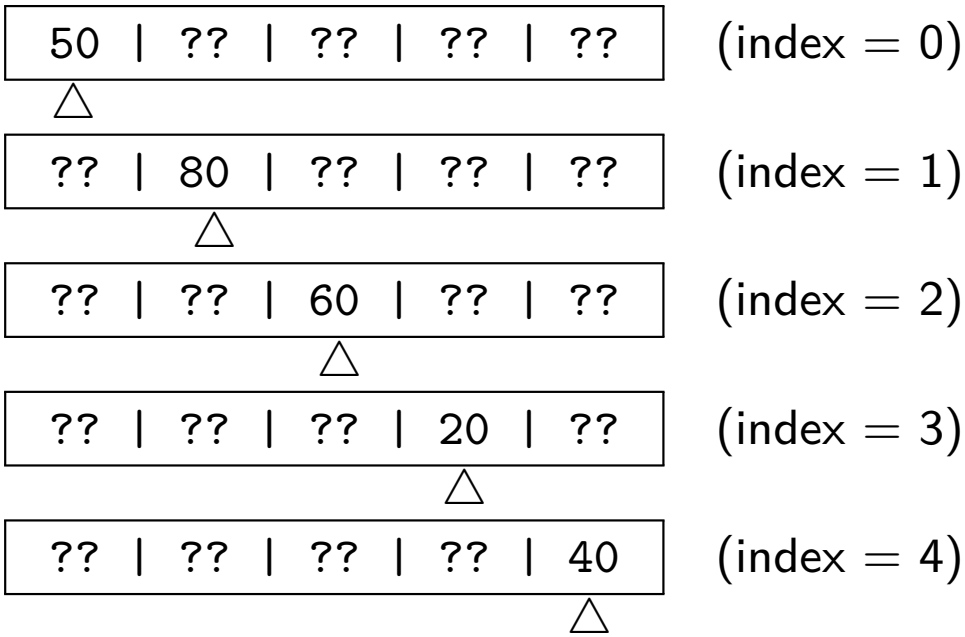
| ?? | 80 | ?? | ?? | ?? | (index = 1)

△

Considérons la liste à trier suivante : | 50 | 80 | 60 | 20 | 40 |

- la **machine** « voit » :
  - $\Rightarrow$ **à travers un (des) index** !

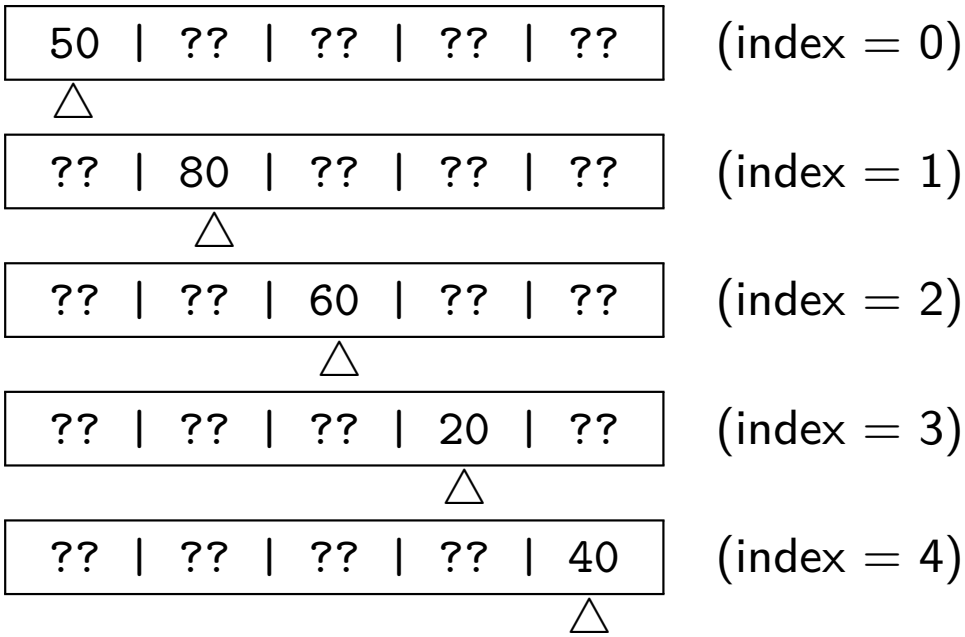
e.g. avec un index :



Considérons la liste à trier suivante : | 50 | 80 | 60 | 20 | 40 |

- la **machine** « voit » :
  - $\Rightarrow$ **à travers un (des) index** !

e.g. avec un index :



- Quid si utilisation de plusieurs index... ?

1 Préambule
- Opération élémentaire : la permutation
- Humain versus Machine

2 Trier une liste
- Classes d'algorithmes
- Tris quadratiques en temps
- Autres tris

Problème

On dispose de n données stockées dans un même ensemble qu'il s'agit d'ordonner de manière croissante.

✎ On peut supposer que ces données sont des nombres entiers.

1 Préambule
- Opération élémentaire : la permutation
- Humain versus Machine

2 Trier une liste
- Classes d'algorithmes
- Tris quadratiques en temps
- Autres tris

Problème

On dispose de n données stockées dans un même ensemble qu'il s'agit d'ordonner de manière croissante.

✎ On peut supposer que ces données sont des nombres entiers.

- Trois classes d'algorithmes en temps d'exécution :

Problème

On dispose de n données stockées dans un même ensemble qu'il s'agit d'ordonner de manière croissante.

✎ On peut supposer que ces données sont des nombres entiers.

- Trois classes d'algorithmes en temps d'exécution :
  1. complexité en $\mathcal{O}(n^2)$ dans le pire des cas et en moyenne

Problème

On dispose de n données stockées dans un même ensemble qu'il s'agit d'ordonner de manière croissante.

✎ On peut supposer que ces données sont des nombres entiers.

- Trois classes d'algorithmes en temps d'exécution :
  1. complexité en $\mathcal{O}(n^2)$ dans le pire des cas et en moyenne
  2. complexité en $\mathcal{O}(n^2)$ dans le pire des cas et en $\mathcal{O}(n \log(n))$ en moyenne

Problème

On dispose de n données stockées dans un même ensemble qu'il s'agit d'ordonner de manière croissante.

✎ On peut supposer que ces données sont des nombres entiers.

- Trois classes d'algorithmes en temps d'exécution :
  1. complexité en $\mathcal{O}(n^2)$ dans le pire des cas et en moyenne
  2. complexité en $\mathcal{O}(n^2)$ dans le pire des cas et en $\mathcal{O}(n \log(n))$ en moyenne
  3. complexité en $\mathcal{O}(n \log(n))$ dans le pire des cas et en moyenne.

Problème

On dispose de n données stockées dans un même ensemble qu'il s'agit d'ordonner de manière croissante.

✎ On peut supposer que ces données sont des nombres entiers.

- Trois classes d'algorithmes en temps d'exécution :
  1. complexité en $\mathcal{O}(n^2)$ dans le pire des cas et en moyenne
  2. complexité en $\mathcal{O}(n^2)$ dans le pire des cas et en $\mathcal{O}(n \log(n))$ en moyenne
  3. complexité en $\mathcal{O}(n \log(n))$ dans le pire des cas et en moyenne.

✎ Les tris les plus intuitifs/naïfs sont en $\mathcal{O}(n^2)$.

✎ La meilleure optimisation possible est dans la 3e classe.

1 Préambule
- Opération élémentaire : la permutation
- Humain versus Machine

2 Trier une liste
- Classes d'algorithmes
- Tris quadratiques en temps
- Autres tris

Tri par sélection

Principe :

- On cherche le minimum dans la liste.
- On permute ce minimum avec le premier élément de la liste.
- On recommence avec la liste restante, jusqu'à épuisement.

Tri par sélection

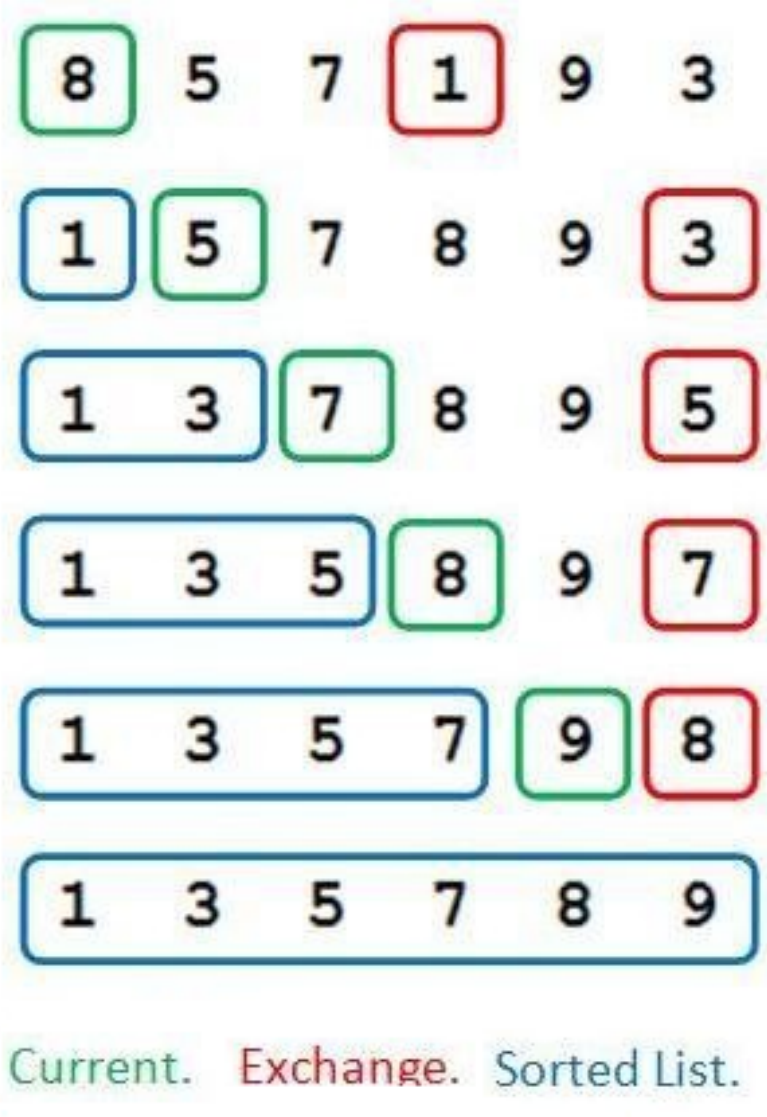
Principe :

- On cherche le minimum dans la liste.
- On permute ce minimum avec le premier élément de la liste.
- On recommence avec la liste restante, jusqu'à épuisement.

✎ On peut, alternativement, prendre le plus grand et l'échanger avec le dernier.

Tri par sélection

Exemple :



Tri par sélection

Algorithme :

```
procédure tri_selection(liste)
    n = longueur(liste)
    pour i = 0 à n-2, faire
        min = i
        pour j = i+1 à n-1, faire
            si liste[j] < liste[min], faire
                min = j
            fin si
        fin pour
        permuter liste[i] avec liste[min]
    fin pour
fin procédure
```

✎ en pseudocode, la borne maximale du `for` (ou autre) est incluse, c'est pourquoi l'on note `n-1` et `n-2` ici, ces bornes correspondent donc respectivement à `n` et `n-1` en Python.

Tri par sélection

Remarques :

- Algorithme intuitif, facile à écrire.
- Complexité :
  - en $\mathcal{O}(n^2)$ (quadratique en temps)
  - toutes les opérations sont à effectuer quelle que soit la liste
    $\Rightarrow$ même complexité dans le pire des cas ou en moyenne.

Tri par bulles

Principe :

- On parcours la liste en échangeant deux éléments consécutifs s'ils sont dans le mauvais ordre.

  $\Rightarrow$ Le plus grand élément de la liste est donc repoussé à la fin.

Tri par bulles

Principe :

- On parcours la liste en échangeant deux éléments consécutifs s'ils sont dans le mauvais ordre.

  ⇒ Le plus grand élément de la liste est donc repoussé à la fin.

  ✎ d'où le nom de l'algorithme
  (les grands éléments sautent par dessus les plus petits jusqu'à la fin de la liste, comme des bulles... qui montent !)

Tri par bulles

Principe :

- On parcours la liste en échangeant deux éléments consécutifs s'ils sont dans le mauvais ordre.

  ⇒ Le plus grand élément de la liste est donc repoussé à la fin.

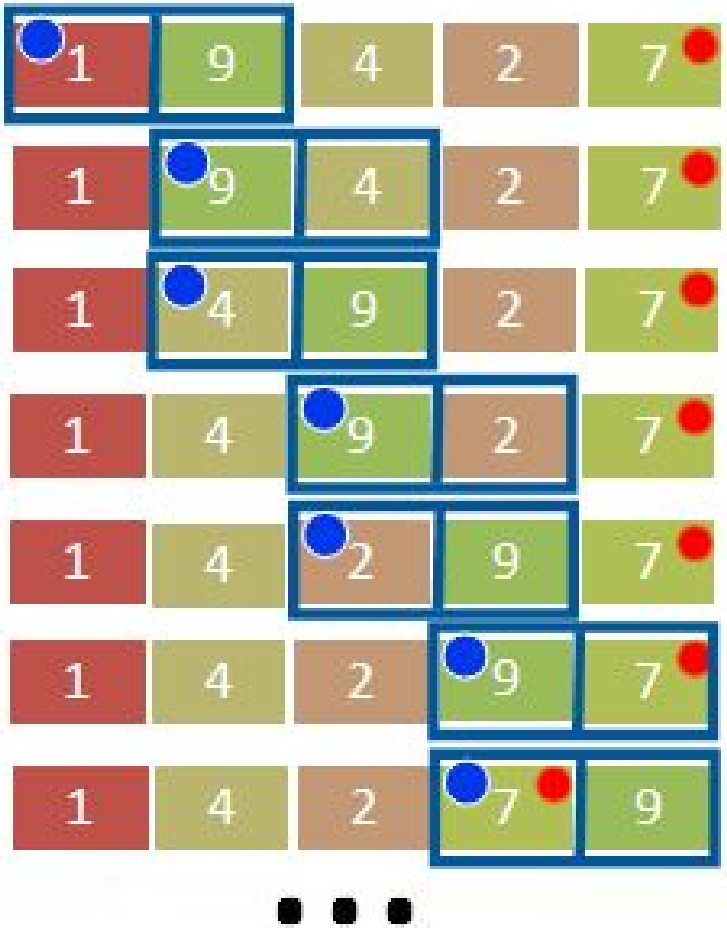
  ✎ d'où le nom de l'algorithme
  (les grands éléments sautent par dessus les plus petits jusqu'à la fin de la liste, comme des bulles... qui montent !)

- Une fois arrivé au dernier élément, on recommence à partir du début, avec les $n - 1$ premiers éléments, et ainsi de suite (le dernier étant fixé car nécessairement le plus grand).

Tri par bulles

Exemple :

- détail du premier parcours uniquement
  - point bleu = position courante
  - point rouge = arrêt du parcours



Tri par bulles

Algorithme :

```
procédure tri_bulle(liste)
    n = longueur(liste)
    pour i = n-1 à 1, faire
        pour j = 0 à i-1, faire
            si liste[j+1] < liste[j], faire
                permuter liste[j] avec liste[j+1]
            fin si
        fin pour
    fin pour
fin procédure
```

Tri par bulles

Algorithme optimisé :

```
procédure tri_bulle(liste)
    n = longueur(liste)
    pour i = n-1 à 1, faire
        permutation = false
        pour j = 0 à i-1, faire
            si liste[j+1] < liste[j], faire
                permuter liste[j] avec liste[j+1]
                permutation = true
            fin si
        fin pour
        si pas de permutation
            fin procedure
    fin pour
fin procédure
```

Tri par bulles

Remarques :

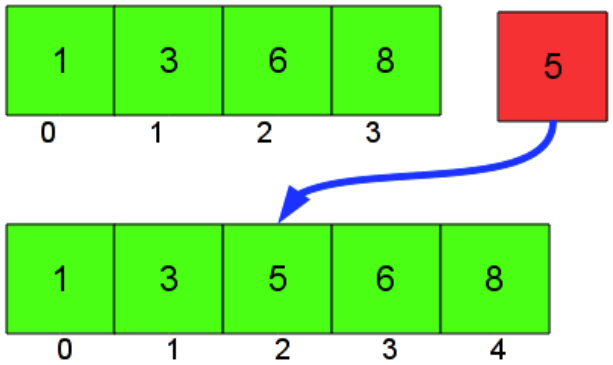
- Algorithme simple à comprendre et à écrire.
- Complexité :
  - en $\mathcal{O}(n^2)$ (quadratique en temps)
  - même complexité dans le pire des cas ou en moyenne.

✎ À noter que si la liste est déjà triée, dès la fin du premier parcours l'algorithme se termine. 

Tri par insertion

Principe :

- On suppose que les $i-1$ premiers éléments de la liste sont déjà classés.
- On insère à sa place le $i^{\text{ème}}$ élément parmi les $i-1$ premiers, de sorte que les i premiers éléments de la liste soient classés.

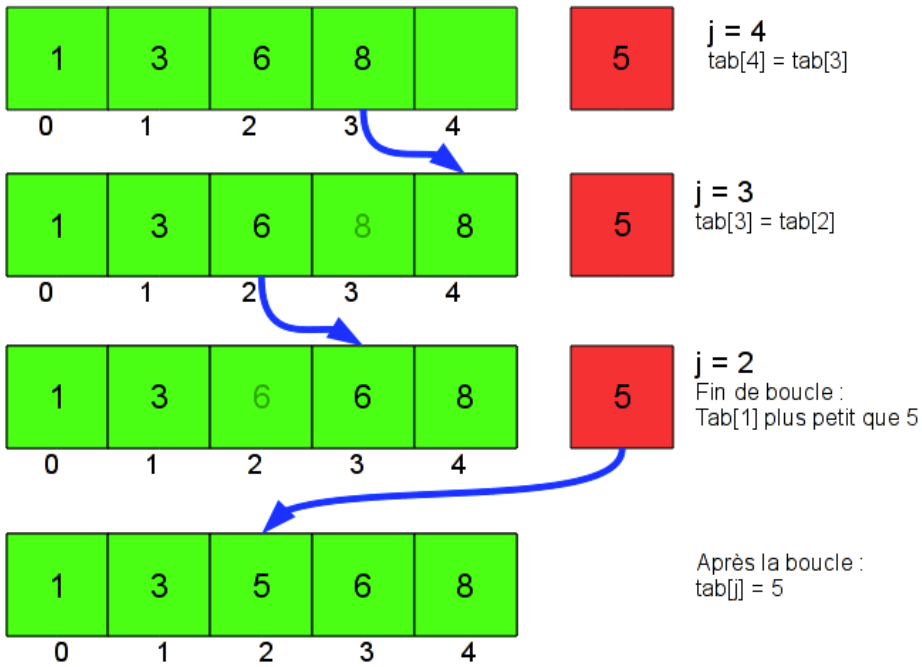


- On recommence jusqu'à insérer le $n^{\text{ème}}$ élément.

Tri par insertion

Détail d'une itération :

- les 4 premiers éléments sont déjà triés, on s'intéresse au 5 ^{ème}.



Tri par insertion

Algorithme :

```
pour i = 1 à n-1, faire
    memoire = liste[i]
    j = i
    tant que (j > 0 et memoire < liste[j-1]), faire
        liste[j] = liste[j-1]
        j = j-1
    fin tant que
    liste[j] = memoire
fin pour
```

Tri par insertion

Remarques :

- Algorithme assez intuitif, moins de comparaisons que le tri par sélection mais plus de permutations.

- Complexité :
  - en $\mathcal{O}(n^2)$ dans le pire des cas
  - en moyenne, temps intuitivement divisé par 2... mais cela ne change pas l'ordre de grandeur : toujours en $\mathcal{O}(n^2)$

1 Préambule
- Opération élémentaire : la permutation
- Humain versus Machine

2 Trier une liste
- Classes d'algorithmes
- Tris quadratiques en temps
- Autres tris

- Approches plus performantes
  - tri rapide (*Quick Sort*)
  - tri par fusion (*Merge sort*)
  - *etc.*

- Certaines approches sont récursives !

- Approches plus performantes
  - tri rapide (*Quick Sort*)
  - tri par fusion (*Merge sort*)
  - *etc.*

- Certaines approches sont récursives !

- Différents tris, algorithmes + vidéos
  - `www.sorting-algorithms.com` (comparaisons possibles)
  - `www.youtube.com`, channel « AlgoRythmics » (folk dance)
  - `https://www.youtube.com/watch?v=ZZuD6iUe3Pc`